



← → ↻

直播實戰 × AI 開發 × MVP

酒Ann

打造你的第一個 互動網站

Vibe coding

實踐指南

上課日期: 12/14(日)、12/21(日)、12/28(日)

時間: 13:30~16:30

- 請先開啟 VS Code並安裝好環境。
- 開啟ChatGPT，建立一個專案Project，名稱為 vibe-coding-week3，於新增檔案裡上傳今日的講義。

在進入部署、後端與商業邏輯之前

我們需要先確認一件最重要的事情：

你現在用的是 什麼身分在寫程式？

這個問題的答案，將決定你今天的學習路徑與思維方式。

Week 3 跟前兩週哪裡不一樣？

Week 1：探索期

- 我能不能照著做出來？
- 這段 code 為什麼會動？
- AI 可不可以幫我補齊？

重點在於理解與實作

Week 2：整合期

- 系統會不會亂掉？
- 狀態、流程、畫面能不能對得起來？
- 我是不是在「堆功能」？

重點在於系統思維

Week 3：交付期

如果這個東西交到別人手上，我不能為它現在的樣子負責？

重點在於產品責任

Software Quality Control Checklist

- ☐ Code review completed
- ☒ Unit tests passed
- ☒ Integration tests passed
- ☒ Security checks completed
- ☒ Performance benchmarks met
- ☒ Documentation updated
- ☒ Deployment approved

評估標準的根本改變

你需要開始思考：

- 如果現在把這個系統交給真實用戶，它會在哪裡出問題？
- 你能不能清楚說明它的邊界與限制？
- 當它壞掉時，你知不知道該從哪裡開始查？

Week 3 的核心任務

所有內容都圍繞在一個目標：

怎麼把 Demo 推到 MVP 的最低門檻



Week 3 的核心任務都圍繞在一個目標：

怎麼把 Demo 推到 MVP 的最低門檻 從這一刻開始 你寫的不是「自己的東西」

請先接受一個現實：只要一個系統符合以下任一條件，它就不再只是你的練習

要給別人使用

不再只是自己看得懂就好，而是要考慮使用者體驗與操作邏輯

要放到網路上

需要考慮安全性、效能、穩定性等真實環境的挑戰

要接 API、資料庫或金鑰

涉及成本、安全、資料一致性等系統性責任

今天的 Demo 策略

1 一個主線 MVP 產品

作為貫穿整週的參考架構

2 多個不同物種的 Web App

展示不同場景下的技術選擇

3 多種「非用後端不可」的情境

理解前後端的責任邊界

這些 Demo 不是要你全部學會，而是要你學會判斷：

- 什麼時候該加後端
- 什麼時候該停
- 什麼時候不該再「只是玩」

今天不只讓你寫更多程式
還會讓你開始知道
什麼樣的程式不該隨便寫

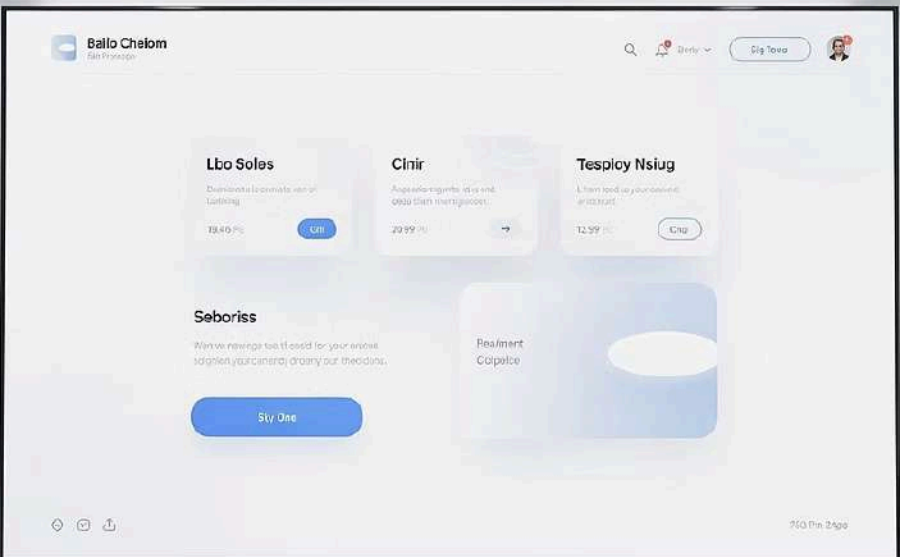
Chapter 1

為什麼 Demo 不夠？

在前兩週，你已經做出「會動的東西」。它們可以展示、可以操作，甚至看起來很完整。

但 Demo 有一個很大的特徵：

- 它只需要對「當下」負責
- 它只需要在「你的電腦」上成立
- 它不需要考慮壞掉之後怎麼辦



什麼叫 MVP (Minimum Viable Product)

MVP 不是「半成品」，也不是「簡單版」。

MVP 的意思是：這是一個已經具備最低可存活條件的產品。

一個 MVP 至少要回答三件事：

1. 誰會用它？
2. 用它要解決什麼問題？
3. 壞了，你知不知道為什麼？

只要有一題答不出來，它就還停留在 Demo 階段。

三個經常被混淆的概念

1 Demo 展示

證明「做得到」

- 可以不穩定
- 可以只在你電腦上運行
- 重點是可行性驗證

2 作品 Portfolio

有完成感的成果

- 可以展示給別人看
- 視覺與功能相對完整
- 但不一定能被真實使用

3 MVP 最小可行產品

具備交付條件

- 有清楚的使用情境
- 有你能負責的範圍
- 壞了你知道為什麼
- 有人「真的用得上」

什麼時候 Demo 應該升級成 MVP

如果你的系統開始出現下面這些情況，就代表它正在跨過那條線：

1 多人使用

不只你一個人會用，需要考慮不同使用者的狀態與權限

2 資料持久化

資料需要被保存，而不是重整就消失

3 行為連續性

使用結果會影響下一步行動，需要狀態一致性

4 可靠性需求

你開始在意「它如果壞了怎麼辦？」

這些都不是技術問題，而是產品已經開始進入真實世界的訊號。

System Architecture



為什麼一升級成 MVP 就會碰到後端

在 Demo 階段，很多事情可以先不管：

- 資料存在記憶體
- 邏輯寫在前端
- 每次重來都沒差

但只要你開始做 MVP，就會立刻遇到：

- 資料要不要留？
- 不同人看到的東西要不要一致？
- 判斷邏輯該放在哪？
- 成本跟風險誰負責？

MVP 不是功能變多，而是責任變清楚

一個常見的誤解是：「我要再加幾個功能，才算 MVP。」

事實正好相反，MVP 的關鍵不是功能數量，而是：

- 有些事你願意負責
- 有些事你現在明確不做
- 哪些風險你已經意識到

❗ 一個功能很多、但責任模糊的系統，比一個功能很少、但邊界清楚的系統，更不像產品。

本週示範的定位

接下來的課程中，你會看到：



一個主線 MVP 產品

作為完整的參考架構，展示從 Demo 到 MVP 的完整轉換過程



多個不同類型的 Web App Demo

涵蓋各種場景，幫助你理解不同情境下的技術選擇



多種「非用後端不可」的情境

清楚展示前後端的責任邊界與分工原則

請記住：這些示範不是要你全部照做，而是要你練習判斷。

Chapter 2

主線 MVP 拆解：節奏型行為追蹤系統



為什麼我們需要一個「主線 MVP」

在 Week 3，你會看到很多不同的 Demo、不同類型的 Web App。如果沒有一個共同參考點，很容易只剩下「看熱鬧」。

因此，我們需要一個主線 MVP，用來回答三個問題：

1. 什麼樣的系統，值得從 Demo 推進成產品？
2. 什麼時候，前端已經不夠？
3. 後端在產品中，到底負責什麼？

 這個主線 MVP，不是為了功能完整，而是為了結構完整。

MVP 的核心定位：追蹤的是「節奏」

這個 MVP 的目的，不是單純記錄你做了什麼。它關心的是：

行為是否穩定

持續性與一致性的觀察

節奏是否斷裂

異常模式的早期偵測

狀態能否維持

長期可持續性的評估

❗ 換句話說，它追蹤的不是「一次結果」，而是一段時間內的變化。這也是為什麼，這個系統一開始就不適合只做成前端 Demo。

先聚焦一個非常單純的問題

如果一個人表面上看起來很努力

但節奏其實一直在崩

我們有沒有辦法提早看出來？

這個 MVP 要做的事情就是：把分散的每日行為，轉成可觀察的狀態，把感覺變成可以討論的依據。

MVP 的最小使用流程

這個系統刻意設計得非常簡單。一個使用者只會做三件事：

1 記錄行為

紀錄今天是否有發生某個行為

2 查看趨勢

查看一段時間內的節奏變化

3 理解狀態

看到系統給出的狀態判斷

沒有設定地獄，沒有複雜操作。因為 MVP 的目的是驗證價值，不是炫耀功能。

為什麼這個 MVP 一定需要後端

如果這個系統只存在前端：

- 資料一重整就消失
- 無法累積歷史紀錄
- 無法比較不同時間區段
- 無法做一致的狀態判斷

這些問題的本質不是技術限制，而是責任放錯地方。

節奏判斷、狀態計算、資料保存，不應該由畫面層來負責。



前端在這個 MVP 中的角色

接收使用者輸入

提供清楚的介面讓使用者記錄每日行為

呈現結果與趨勢

用視覺化方式展示時間序列資料與狀態變化

不做任何關鍵判斷

所有狀態計算與節奏分析都由後端負責

① 前端的任務是讓人「看得懂現在的狀態」，而不是決定狀態本身。

後端在這個 MVP 中的角色

儲存時間序列資料

確保資料的持久性與可回溯性。

計算節奏指標

進行一致性分析與模式識別。

回傳狀態結果

提供經過處理的判斷結果。

① 這些工作有一個共同特徵：它們需要一致性、可回溯性、可調整性。這也是為什麼，後端在 MVP 階段就已經是必要角色。

MVP 的「狀態」設計比功能更重要

這個 MVP 不追求大量數字。它刻意把結果收斂成狀態，例如：

1 穩定

節奏一致，風險低

2 波動

出現不規則變化

3 斷裂風險

可能無法持續

這樣的設計，讓系統可以：

- 支援不同使用情境
- 被人快速理解
- 成為決策參考，而不是資料堆疊

這個 MVP 的最低完成標準

Checklist

- ☒ Task 1
- ☐ Task 1
- ☐ Task 2
- ☒ Task 3
- ☐ Task 3

前端必須做到：

- 能讓使用者記錄每日行為
- 能顯示一段時間內的變化
- 不在前端做任何節奏或狀態判斷。

刻意不需要做到：

- 使用者登入
- 權限管理
- 複雜設定
- 商業化流程

後端必須做到：

- 能儲存每日行為紀錄
- 能依時間回傳資料
- 能回傳一個「狀態結果」，而不是一堆數字。

關鍵判斷基準

問題如果跟「一致性、判斷、責任」有關
那它通常不屬於前端

Chapter 3

前端與後端 不是技術選擇，是責任切分

在軟體開發的世界裡，前端與後端的分工不僅僅是技術架構的選擇，更是系統責任與可靠性的基礎。



從 Demo 到 MVP：模糊地帶的終結

Demo 階段的自由

- 邏輯可以直接寫在畫面裡
- 資料暫存在記憶體中
- 重新整理就重新開始
- 一切都是暫時的、可變動的

MVP 階段的要求

- 邏輯必須有明確歸屬
- 資料需要持久化儲存
- 狀態必須可以恢復
- 責任界線必須清晰

當系統從概念驗證進入最小可行產品階段，這些曾經可以模糊處理的地方，都會成為系統風險的來源。

① 前端與後端的分工，正是為了讓責任清楚、系統可靠。

如果這件事做錯了，影響的是「畫面呈現」，還是「系統判斷結果」？

- 使用者體驗
- 視覺回饋
- 互動流程

- 商業邏輯
- 資料一致性
- 系統決策

這個簡單但強大的原則，將在後續所有 Web App 開發案例中反覆驗證其價值。

前端不該承擔的責任

一旦系統進入 MVP 階段，有些責任絕對不應該由前端承擔。這不是能力問題，而是架構原則：

判斷使用者狀態

使用者的權限、身份、狀態應該由後端統一管理與驗證

計算關鍵指標

影響商業決策的數據計算必須在後端完成，確保準確性與一致性

決定通過或不通過

任何會影響系統行為的判斷邏輯都應該集中在後端處理

保存長期歷史資料

持久化資料必須由後端負責，前端只負責暫時的 UI 狀態

前端承擔後端責任的三大風險

1 每個人看到的結果可能不同

當判斷邏輯在前端執行時，不同裝置、不同瀏覽器、不同版本可能產生不同的結果，導致資料不一致。

2 重整或換裝置就失效

前端的狀態是暫時性的，一旦使用者重新整理頁面或切換裝置，所有計算與狀態都會消失。

3 商業邏輯暴露在畫面層

關鍵的商業規則與判斷邏輯如果寫在前端，不僅容易被繞過，也無法有效保護智慧財產。

後端存在的真正理由

後端的存在，不是因為技術比較複雜或比較難，而是因為有些事情必須集中處理。這些事情的共同特徵是：一旦出錯，後果不只影響單一使用者。



資料儲存：永久保存系統資料



一致性：確保所有人看到相同結果



判斷邏輯：執行商業規則與決策



可回溯性：追蹤歷史變更與狀態

案例分析：主線 MVP 的前後端分工

讓我們用一個具體的每日行為追蹤系統來說明前後端的責任分工：

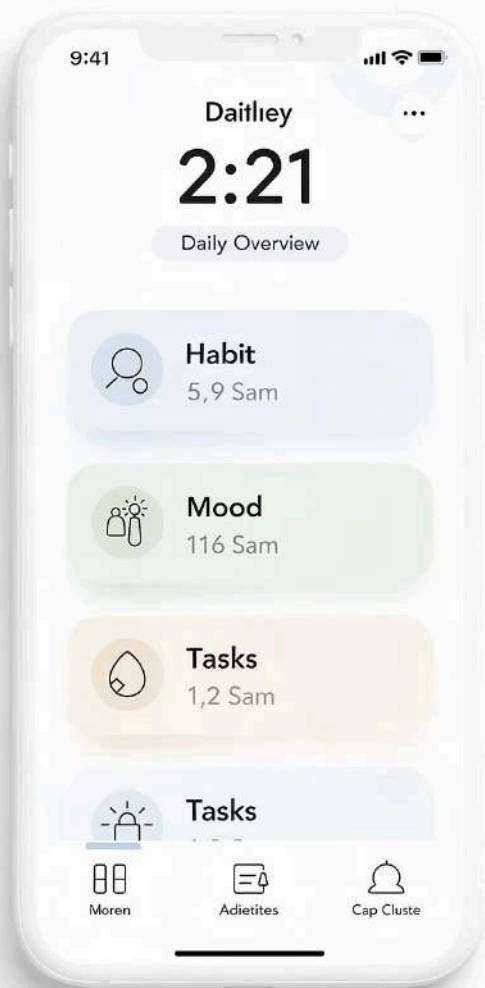
前端負責

- 提供介面讓使用者記錄每日行為
- 顯示節奏變化的視覺化圖表
- 即時反應使用者的操作
- 提供流暢的使用體驗

後端負責

- 持久化儲存每日紀錄到資料庫
- 計算使用者的行為節奏
- 判斷當前狀態並回傳結果
- 維護資料的一致性與完整性

❗ 關鍵思考：如果節奏判斷寫在前端，那麼每一個裝置都會活在不同版本的世界裡，無法確保資料的正確性。



為什麼「一致性」必須是後端責任



多人使用

不同使用者必須看到一致的資料與結果



跨裝置使用

同一使用者在不同裝置上應該看到相同狀態



歷史回溯

必須能夠追蹤並恢復過去的狀態與資料

只要系統符合以上任一條件，「一致性」就不再是 UI 層面的問題。



一致性意味著：同一筆資料、同一套判斷、同一個結果。這些條件，前端無法獨立保證。

常見錯誤：把後端邏輯藏在前端

這種做法在初期看起來很快、很方便，畢竟不用處理後端的複雜度。但這會帶來三個嚴重的後果：

→ 無法擴充

當需求變更或功能擴展時，前端邏輯會變得越來越複雜且難以維護，最終需要大規模重構。

→ 無法除錯

問題發生時，你無法確定是哪個裝置、哪個版本、哪個狀態造成的，除錯變成噩夢。

→ 無法說清責任

當出現資料錯誤或商業邏輯問題時，你無法明確界定是前端還是後端的責任，團隊協作會陷入混亂。



經驗法則：

當你開始覺得「這段邏輯有點重要」，那它通常已經不適合放在前端了。

哪些 Web App 一定需要後端？

在接下來的課程中，你會看到多種不同類型的 Web App。

它們的共同點不是功能複雜，而是**責任無法分散在畫面層**。

即時同步系統

多個使用者需要看到相同的即時資料，例如協作編輯工具、聊天室。

決策型系統

系統需要根據規則做出判斷，例如審核系統、推薦引擎。

長期記憶型系統

需要保存並分析歷史資料，例如行為追蹤、報表系統。

判斷責任歸屬的三個關鍵問題

1 這個功能如果寫在前端，會不會出事？

思考最壞的情況：如果使用者清除快取、換裝置、或有人試圖繞過邏輯，會發生什麼問題？

2 這個判斷如果每個人都不一樣，後果是什麼？

評估不一致性的影響：是否會造成資料錯誤、商業損失、或使用者體驗破碎？

3 這個責任，我現在有沒有能力承擔？

誠實評估團隊的技術能力與時間資源，不要因為短期便利而埋下長期風險。

如果你能清楚回答這些問題，代表你已經具備進入後端與 MVP 實作的心理準備。

前後端分工的成熟度階梯

1 階段一：全寫在前端

Demo 階段可以接受，但缺乏可靠性與擴展性。

3 階段三：基於責任分工

能夠根據「影響畫面」或「影響判斷」來決定歸屬。

2 階段二：開始意識到分工

知道應該分離，但還不確定如何劃分界線。

4 階段四：系統性思考架構

在設計初期就能規劃清楚的責任邊界與 API 介面。

實務案例：會員系統的责任分配

前端負責

- 收集使用者的帳號密碼
- 顯示登入狀態與錯誤訊息
- 提供登出按鈕與互動
- 暫存 UI 層面的使用者資訊

後端負責

- 驗證帳號密碼是否正確
- 產生並管理 Session Token
- 檢查使用者權限與狀態
- 記錄登入歷史與安全事件

❗ 千萬不要在前端判斷「密碼是否正確」或「使用者是否有權限」，這些都是後端的核心責任。

實務案例：購物車系統的責任分配



價格計算、折扣規則、庫存驗證這些關鍵邏輯，絕對不能只在前端處理。即使前端先顯示預估金額，最終的計算與驗證必須由後端完成。

資料流向：前後端的協作模式

1 使用者輸入

在前端介面中操作

2 前端驗證

基本格式檢查

3 送至後端

透過 API 傳遞資料

4 後端處理

驗證、計算、儲存

5 回傳結果

返回處理結果

6 前端顯示

呈現給使用者

這個流程確保了資料的正確性、安全性與一致性。

前端的驗證只是為了提升使用者體驗，後端的驗證才是真正的守門員。

MVP 階段的架構檢查清單



資料持久性

關鍵資料是否都儲存在後端？重新整理後資料是否完整？



商業邏輯保護

判斷規則是否在後端執行？前端是否可以被繞過？



一致性保證

多裝置、多使用者是否看到相同結果？



可回溯性

重要操作是否有紀錄？能否追蹤歷史狀態？

在進入 MVP 開發之前，用這個清單檢查你的架構設計，可以避免很多後續的問題。

技術債的預防

正確的分工是投資

短期視角

把邏輯寫在前端看似更快：

- 不用學後端技術
- 不用處理 API 介接
- 不用設定伺服器與資料庫
- 可以立即看到結果

長期視角

正確的分工是對未來的投資：

- 系統更容易擴展與維護
- 團隊協作更加清晰有效
- 資料安全性與一致性有保障
- 重構成本大幅降低

i 許多團隊在 MVP 階段為了速度而犧牲架構，最後花費數倍的時間重構。正確的分工從一開始就是必要的投資。



Chapter 4

BYOK 是產品設計問題

在現代產品開發中，BYOK（Bring Your Own Key）不只是技術選擇，更是決定產品生死的關鍵設計決策。



什麼是 BYOK？

核心概念

BYOK (Bring Your Own Key) 是指使用者在使用系統時，必須自己提供 API Key，例如 AI 服務或第三方 API 的金鑰，而不是由產品方統一代付或代管所有使用者的呼叫成本。

三個關鍵影響面向

- 成本計算與分攤方式
- 風險責任歸屬
- 產品長期存續能力

📌 這個看似簡單的技術選擇，實際上會深刻影響整個產品的商業模式與風險結構。

BYOK 的三大核心影響

1 成本怎麼算

決定每次 API 呼叫的費用由誰承擔，影響產品的定價策略與獲利模型。

2 風險誰扛

明確劃分金鑰管理、濫用防範、資安事件等責任歸屬。

3 能不能活下來

直接決定產品在用量成長時，是否會因成本失控而無法持續營運。

Demo 階段的盲點

在產品的 Demo 階段，開發者常常採取「先求有再求好」的做法：使用自己的 API Key 來快速驗證概念、流量很小所以成本可控、只有內部團隊在使用。

這些做法在概念驗證階段完全合理，但一旦進入 MVP 階段，系統開始真正被外部使用者使用時，問題就會迅速浮現。



為什麼 Demo 階段容易忽略 BYOK

1 先用自己的 Key 撐著

開發者使用個人或團隊的 API Key，快速讓功能跑起來，專注於驗證核心邏輯。

2 使用者範圍受限

只有團隊成員或少數測試者在使用，金鑰管理問題尚未浮現。

3 流量規模很小

呼叫次數少，成本幾乎可以忽略，暫時感受不到壓力。

❗ 關鍵轉折：

從 Demo 進入 MVP，你不再只是在證明「跑得動」，而是要讓系統「被使用」。這時 BYOK 議題必須正面處理。

不做 BYOK 會發生什麼事？

如果選擇由產品方統一管理金鑰

假設你決定讓所有使用者共用一把 Key，或將金鑰直接寫在系統程式碼中，你將立刻面臨以下風險：

- 成本爆炸：用量增加時，API 費用會線性甚至指數成長
- 無法追蹤：當系統出問題時，難以釐清是哪個使用者造成的
- 責任全擔：金鑰外流或濫用時，所有後果由產品方承擔

這些風險不會因為你只是在做 MVP 就自動消失。

BYOK 的本質：責任切分

BYOK 真正在解決的核心問題是：這個系統所產生的成本與風險，到底是「使用者的責任」，還是「產品擁有者的責任」？

當你要求使用者帶自己的 Key 時，你就清楚地將成本與濫用風險的責任劃分出去。

① 這不是逃避責任，而是建立一個可持續運作的商業模式。

三種 BYOK 設計方式概覽



完全前端 BYOK

金鑰由使用者在前端輸入，前端直接呼叫第三方服務。最簡單但風險最高。



後端轉接型 BYOK

使用者提供 Key，由後端代為呼叫。平衡安全性與複雜度，最適合 MVP 階段。



平台代管型

產品方統一吸收成本。適合成熟商業產品，但不適合 MVP 階段。

方案一：完全前端 BYOK

運作方式

使用者在網頁介面直接輸入 API Key，
JavaScript 程式碼在瀏覽器端直接呼叫第三方
API，不經過任何後端伺服器。

適用場景

- 個人工具或學習專案
- 技術展示作品
- 非正式使用情境

優點

- 實作結構最簡單
- 成本與責任完全在使用者
- 不需要維護後端伺服器

限制與風險

- 金鑰完全暴露在瀏覽器
- 無法進行呼叫次數限制
- 不適合多人協作環境
- 難以整合進商業產品

方案二：後端轉接型 BYOK

最適合 MVP 階段的選擇

使用者提供 API Key，但由後端伺服器代為呼叫第三方服務。這是在安全性、複雜度與控制力之間取得最佳平衡的方案。

核心優勢

- 金鑰不會暴露在前端程式碼中
- 可以加上呼叫頻率限制
- 便於監控與除錯
- 系統架構仍然可控

需要承擔的責任

- 維護最小的系統穩定性
- 設計錯誤處理機制
- 實作基本的限流保護

這是本課程在 MVP 階段最推薦的實作方式。

後端轉接型的技術架構

1 使用者提供 Key

前端介面讓使用者輸入並暫存自己的 API Key（通常存在 LocalStorage）。

3 後端代為呼叫

後端使用收到的 Key 呼叫第三方服務，並加上限流、驗證等保護機制。

2 後端接收請求

使用者發起功能請求時，將 Key 隨請求一併傳送到後端 API。

4 回傳處理結果

後端將第三方服務的回應處理後，回傳給前端顯示。

方案三：平台代管型

運作模式

所有使用者共用平台的 API Key，由產品方統一吸收所有呼叫成本。這是許多成熟 SaaS 產品採用的方式。

為什麼不適合 MVP？

- 成本完全不可控
- 需要複雜的計費與限額系統
- 濫用風險直接由產品方承擔
- 需要充足的資金與商業準備

 **重要提醒：**這不是 MVP 階段該碰的設計方式。除非你已經有明確的商業模式與資金支持。

三種方案的比較總結

特性	完全前端	後端轉接	平台代管
實作複雜度	低	中	高
金鑰安全性	低	高	高
成本承擔者	使用者	使用者	產品方
控制與監控	無	有	完全掌控
MVP 適用性	僅限個人專案	最推薦	不建議
商業化潛力	低	中高	高

為什麼 BYOK 必須搭配後端設計

一旦選擇「後端轉接型 BYOK」，後端就不再只是單純的資料存取層，它必須扮演關鍵的中介與保護角色。



安全地接收與使用 Key

確保金鑰在傳輸與暫存過程中不被洩漏。



控制呼叫頻率

實作限流機制，防止單一使用者濫用資源。



處理錯誤與回傳

統一處理第三方 API 的錯誤，回傳友善的訊息給前端。

這些責任前端無法也不應該承擔，這也是為什麼 BYOK 與後端設計是緊密綁定的議題。

本課 Web App 中的 BYOK 情境

在接下來的實作章節中，你將會遇到多種涉及 BYOK 的應用情境：

1 AI 輔助型功能

例如使用 OpenAI API 進行文字生成、摘要或分析，每次呼叫都會產生費用。

2 決策型 Web App

透過第三方 API 取得即時數據或外部資訊，輔助使用者做出決策。

3 分析與建議系統

整合多個付費服務，提供深度分析或客製化建議。

BYOK 與商業評估的關係

如果你在評估一個產品時，不知道每個使用者會花多少錢、不知道成本落在誰身上、不知道流量放大後會發生什麼事，那任何商業判斷都只是建立在假設之上。

BYOK 不是可有可無的技術細節，它是商業評估的前置條件。

沒有清楚的成本結構，就無法做出正確的定價策略與成長規劃。



商業評估失真的四個常見場景

低估使用者成本

以為每個使用者只會產生幾元成本，實際上可能高達數十元甚至更多。

忽略濫用風險

沒有限流機制，單一使用者可能在短時間內耗盡整月預算。

成長曲線失真

用量成長 10 倍時，成本也跟著成長 10 倍，但收入並未同步增加。

責任歸屬不明

出現資安事件或濫用情況時，不清楚該由誰承擔後果。

Chapter 5

Web App 實作
用 Q 做出可部署的 MVP



你要做的不是練習，是 MVP

從這一章開始，你的目標不再只是「寫出會動的東西」，而是要完成一個可以被其他人透過網路連線使用的真實 Web App。

1 前後端真的接在一起

不是兩個獨立的程式，而是透過 API 溝通的完整系統。

2 系統可以部署到雲端

不只在本地跑，還要能夠讓任何人透過 URL 存取。

3 你說得出每一層的責任

清楚理解前端、後端、資料庫各自負責什麼功能。

 **重要：**本章不會提供完整程式碼，你將透過 Developer Q 指令，一步一步把系統「生出來」。

實作方法：嚴格的六步驟流程

每一個 Web App 都必須嚴格按照以下順序完成，只要順序亂掉，卡關機率會大幅提高：

1 建立專案骨架

用 Q 產生資料夾結構與空檔案

2 定義 API 契約

用 Q 設計前後端溝通的規格

3 產出後端程式

用 Q 逐檔案生成後端邏輯

4 產出前端程式

用 Q 生成介面與呼叫邏輯

5 前後端合體

用 Q 協助整合與除錯

6 雲端部署

用 Q 完成上線與測試

三種部署方式選擇

選項 A：Cloud Run + 靜態前端

後端使用 Cloud Run 部署容器化應用，前端可用任意靜態託管服務。最接近真實產品環境，最推薦。

選項 B：Firebase Hosting + Cloud Functions

完全使用 Firebase 生態系，前端用 Hosting，後端用 Functions。對新手最友善。

選項 C：Firebase Hosting + Cloud Run

混合式架構，結合 Firebase 的前端便利性與 Cloud Run 的後端彈性。實務常見組合。

 注意：Firebase Hosting 只能放前端靜態檔案，無法直接部署後端 API。

Web App #1

主線 MVP：節奏型行為追蹤系統

這是本課的核心實作案例，必須完成。透過這個專案，你將學會完整的前後端整合與部署流程。

節奏型行為追蹤系統的最低完成標準

這個 Web App 的 MVP 只需要做到以下四件事，不需要複雜的功能或華麗的介面：

1 前端可以送出每日紀錄

提供簡單表單，讓使用者輸入日期與是否完成行為。

2 後端可以儲存資料

接收前端送來的資料，存入記憶體或簡單檔案。

3 後端會回傳狀態

根據資料計算出連續天數或中斷狀態。

4 前端只顯示結果

接收後端回傳的狀態，在畫面上呈現，不做任何判斷邏輯。

做到這四點，就已經是一個完整的 MVP。

主線 MVP 完成標準檢核

當你完成上述所有步驟後，請確認自己能夠回答以下問題：

後端部署在哪裡？

你應該能說出是 Cloud Run 還是 Firebase Functions，以及專案名稱。

API 的公開 URL 是什麼？

完整的網址，可以直接用 curl 或 Postman 測試。

前端如何連到後端？

說明 API_BASE_URL 的設定位置與運作方式。

為什麼 status 不能在前端算？

理解業務邏輯應該在後端的原因與重要性。

如果你能清楚回答這些問題，恭喜你完成了第一個真正的 Web App MVP！

Web App #2 與 #3：延伸實作

接下來還有兩個 Web App 案例供你選擇實作：

Web App #2：匿名回饋牆

一個讓使用者匿名留言與回覆的系統，練習處理多筆資料與動態更新。

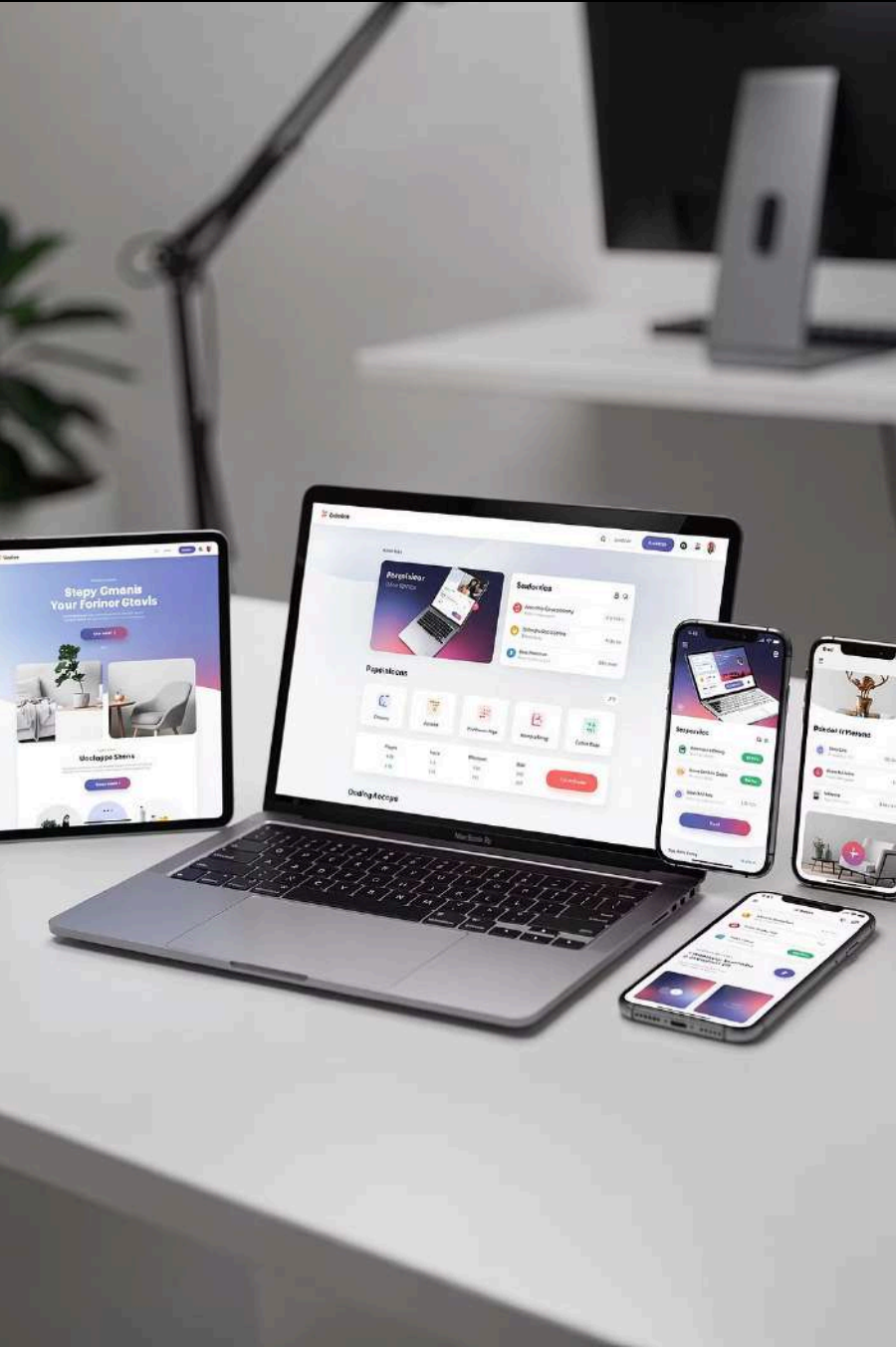
- 留言的新增與讀取
- 簡單的內容過濾
- 時間排序與分頁

Web App #3：決策型 Web App

整合外部 API，協助使用者做出決策或取得建議。這裡會真正涉及 BYOK 設計。

- 後端轉接型 BYOK 實作
- 呼叫第三方 AI 服務
- 錯誤處理與限流

這兩個案例完全沿用相同的六步驟流程，只是換需求、換 API 契約、換部署選擇。你不需要學新技術，只是在練習「產品型實作節奏」。



實作過程中的常見卡關與解法

CORS 錯誤

前端無法呼叫後端 API。解法：在後端加上正確的 CORS header，允許前端網域存取。

API 契約不一致

前端送的資料格式與後端期待的不同。解法：回到 Step 2 檢查契約，確保雙方理解一致。

部署後無法連線

本地可以跑，雲端版本卻失敗。解法：檢查環境變數、防火牆設定、雲端服務的權限。

前端顯示異常

後端有回傳資料，但前端沒正確顯示。解法：用瀏覽器 DevTools 檢查 console 與 network tab。

用 Q 驅動開發的核心技巧

成功使用 Developer Q 完成 Web App 的關鍵，在於你給出的指令品質與流程控制。



明確指定範圍

每次只要求 Q 產出一個檔案或完成一個小任務，避免一次要求太多導致輸出混亂。



要求文件而非程式

在設計階段（如 API 契約），明確要求「文件格式」，避免 Q 過早產出程式碼。



建立檢核點

每完成一個步驟，先測試是否正常，再進入下一步。不要一次跑完所有流程。



追問細節

如果 Q 的回答不夠具體，直接追問「這個參數是什麼意思」、「為什麼要這樣設計」。

從 MVP 到產品的進階思考

完成 MVP 只是第一步，真正的產品還需要考慮許多面向：

- 1 資料持久化：從記憶體或檔案儲存升級到真正的資料庫（如 Firestore、PostgreSQL）。
- 2 使用者認證：加入登入系統，讓每個使用者有自己的資料空間。
- 3 錯誤處理：完善的錯誤訊息與重試機制，提升系統穩定性。
- 4 效能優化：加入快取、分頁、索引等機制，應對更大的流量。
- 5 監控與日誌：追蹤系統運作狀況，快速發現並修復問題。

BYOK 在實際產品中的延伸議題

進階挑戰

- 金鑰加密儲存：如果要暫存使用者的 Key，如何安全地加密？
- 限流與配額：如何防止單一使用者在短時間內耗盡 API 額度？
- 多種 API 整合：如何讓使用者選擇不同的 AI 服務提供商？
- 成本透明化：如何讓使用者清楚知道每次操作會花多少錢？

這些都是從 MVP 走向成熟產品時，必須認真思考與設計的問題。

你真正學會的核心能力

完成本章的實作後，你獲得的不只是某個特定框架或語言的技能，而是一套可遷移的產品開發方法論：

需求轉指令

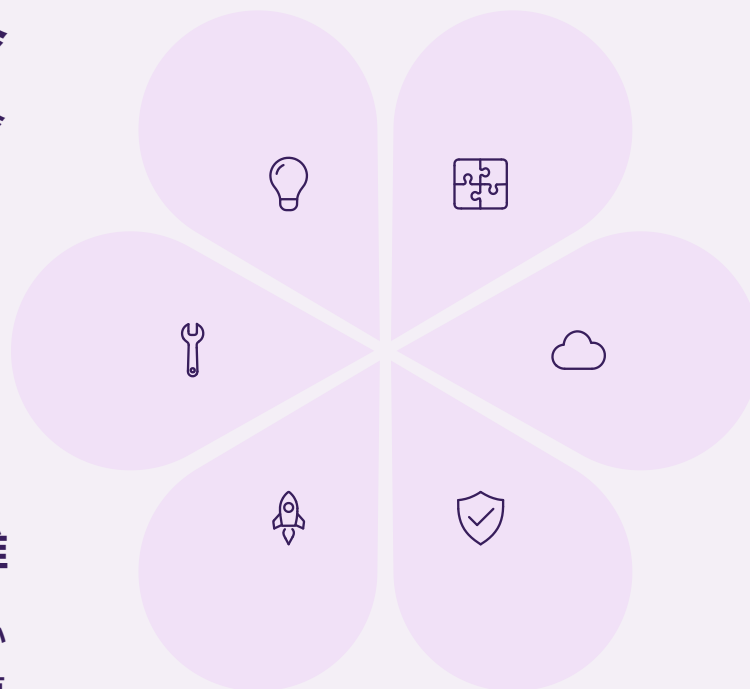
把產品需求拆解成清晰的 AI 指令

工具運用

善用 AI 協作工具提升開發效率與品質

MVP 思維

專注於最小可行版本，快速驗證核心價值



前後端協作

理解 API 契約在系統整合中的關鍵角色

雲端部署

將本地開發的系統上線到公開網路

責任切分

透過 BYOK 設計做出正確的成本與風險決策

Chapter 6

Git 同步與交付

讓你的 Web App 不是只活在你電腦

從個人專案到可交付產品，讓我們一起建立專業的協作與部署流程



為什麼「能跑」不等於「完成」？

在 Chapter 5，你已經讓前端、後端與部署全都運作起來。這是一個重要的里程碑，但並不代表專案真正完成。

一個完整的產品不只是能在你的電腦上運行，更要能被其他人接手、部署與維護。

許多開發者在這個階段會遇到一個常見的盲點：專案在自己的開發環境中運作良好，卻無法順利交付給團隊或部署到生產環境。

❗ 這不是技術能力的問題，而是缺乏系統化的交付流程。

四個常見的「未完成」警訊

環境依賴問題

專案在你的電腦上能跑，但換到另一台電腦就出現各種錯誤訊息，可能是路徑、套件版本或環境變數設定的問題

啟動流程不明

團隊成員拿到專案後不知道該執行哪些指令、安裝哪些相依套件，或是設定哪些環境變數才能啟動

關鍵人員依賴

只有你知道如何處理專案中的特殊設定或問題，一旦你不在場，其他人就無法獨立解決問題

無法回溯修改

當程式碼出現問題時，沒有清楚的版本記錄可以追蹤，也不知道該回到哪個穩定的版本

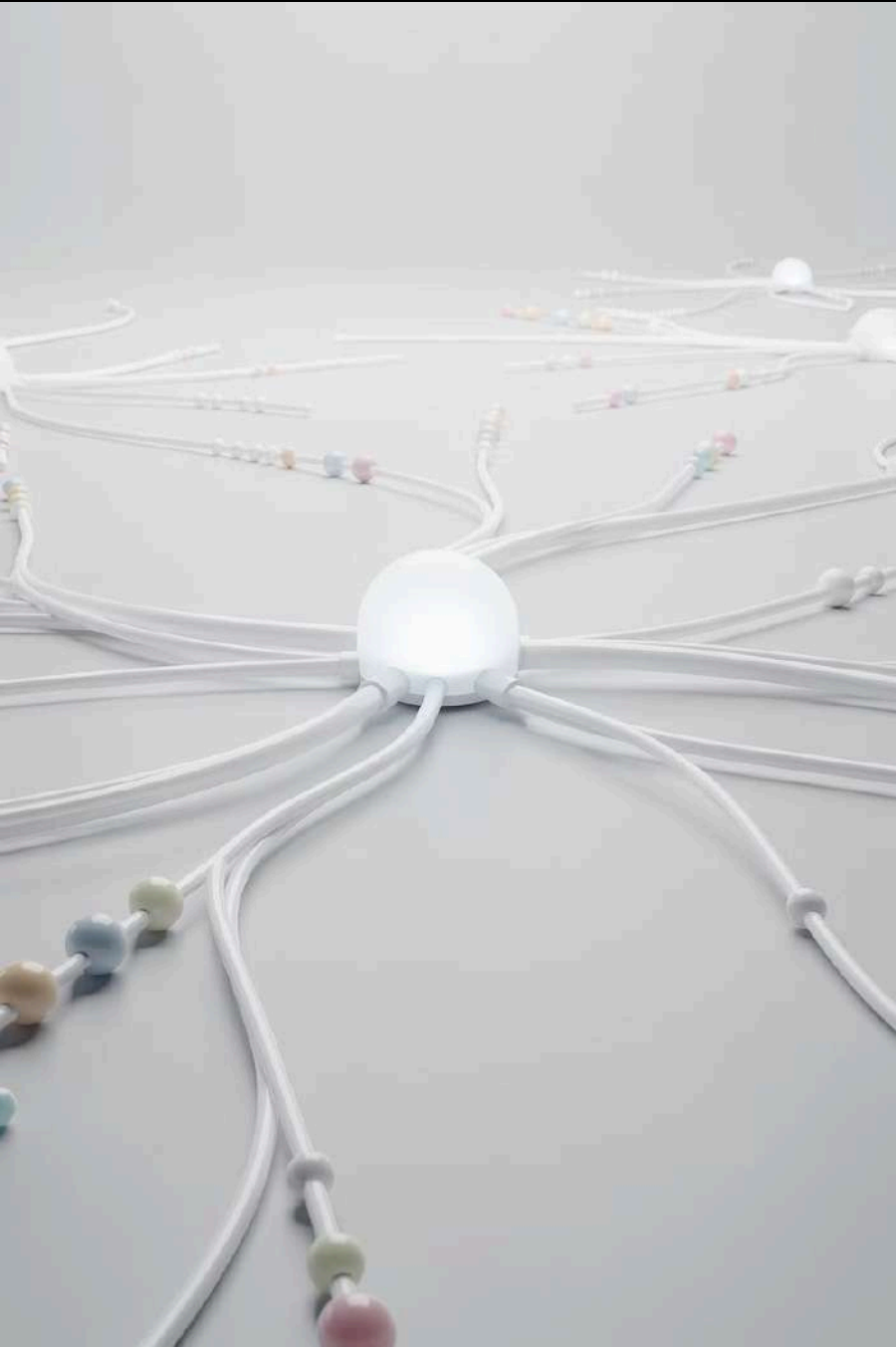
Git 的真正價值：建立交付結構

不只是版本控制工具

許多初學者把 Git 當作單純的備份工具，但 Git 的真正價值在於建立一個完整的交付結構。透過 Git，你可以：

- 記錄每一次有意義的程式碼變更
- 在團隊中同步最新的開發進度
- 當出現問題時快速回到穩定版本
- 讓新成員能順利接手專案

這一章的目標不是讓你成為 Git 專家，而是建立基本但完整的專案交付能力。



不再是單點故障

當你完成這一章，你的專案將不再完全依賴你個人。團隊中的任何人都能接手、部署與維護，這才是一個真正可交付的產品。



使用 Developer Q 協助

在這個步驟中，我們將使用 AI 助理來協助檢查與優化專案結構。這是一個很好的學習方式，因為 AI 可以快速指出常見的問題與最佳實踐。



向 Developer Q 提問的完整 Prompt

你是 Git 與交付顧問。我現在有一個全端 Web App 專案（frontend/ + backend/）。

請協助我檢查並優化專案結構，使它適合被同步到 Git 與交付給他人。

請輸出：

1. 建議的專案根目錄結構（包含 frontend、backend、README）
2. 哪些檔案「一定不能 commit」
3. 哪些檔案「一定要有」
4. 常見新手在交付時會漏掉的項目清單

請用檢查清單形式輸出，不要貼程式碼。

將這段文字複製到 Developer Q 中，它會根據你的專案類型提供客製化的建議。

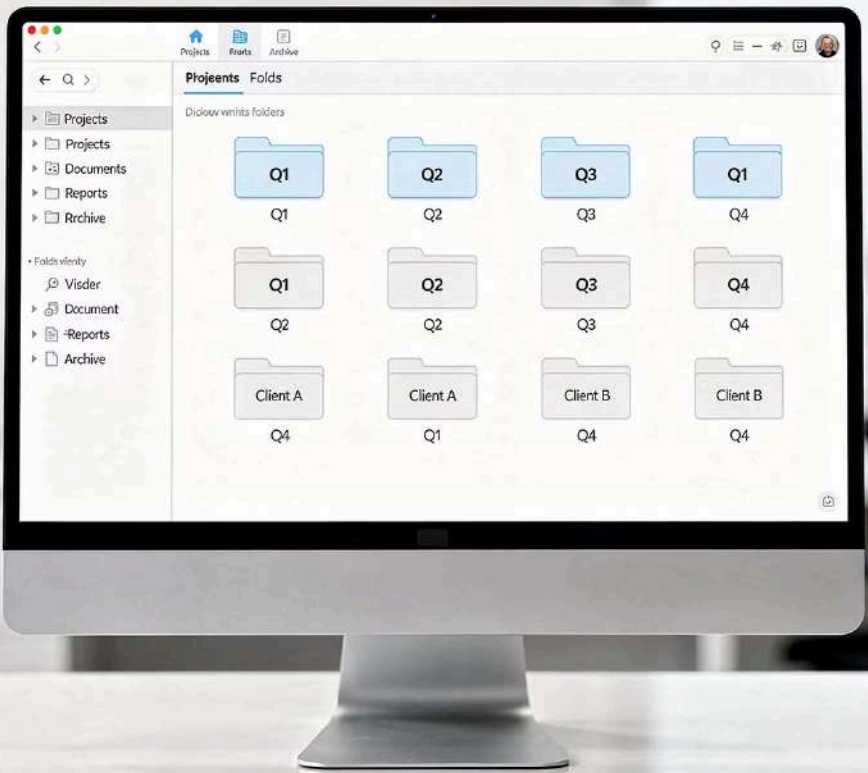
檢查清單：對照你的專案結構

收到 AI 的回應後，請仔細對照你目前的專案結構。重點不是要完全符合某個標準範本，而是確認以下幾個關鍵點：

- 前端與後端的資料夾是否清楚分離
- 專案根目錄是否有 README 文件
- 是否有設定檔被放在不適當的位置
- 資料夾結構是否能让不熟悉專案的人快速理解

如果發現問題，現在就是整理的好時機。

i 調整資料夾結構比調整程式碼容易得多，而且對長期維護有很大的幫助。



使用 Q 產生適合的 .gitignore

請根據一個「純前端 + Node 後端」的 Web App，幫我產出一份合適的 .gitignore 建議清單。

需求：

- 不要 commit API key、環境變數
- 不要 commit build 或暫存檔
- 請逐條解釋「為什麼要忽略」

請用「註解說明版」輸出。

- ❗ 這個 prompt 會讓 AI 不只給你一份 .gitignore 檔案，還會解釋每一行的用途。這對學習非常有幫助，因為你會理解為什麼要忽略這些檔案。

常見應該被忽略的檔案類型

1 環境變數與設定檔

.env 檔案、config.json 等包含敏感資訊的設定檔，這些檔案通常包含 API 金鑰、資料庫連線字串等機密資訊

2 相依套件資料夾

node_modules/ 資料夾，這個資料夾可能包含數千個檔案，而且可以透過 package.json 重新安裝

3 建置產出檔案

dist/、build/ 等資料夾，這些是編譯或打包後的結果，可以從原始碼重新產生

4 IDE 與系統檔案

.DS_Store、.vscode/ 等編輯器或作業系統產生的檔案，這些檔案與專案本身無關

5 日誌與暫存檔

.log、.tmp 等執行時產生的暫時性檔案，這些不需要被版本控制

README 必須包含的七個段落

1 專案簡介

用一段話（最多三句）清楚說明這個專案在做什麼、解決什麼問題

2 系統架構

說明前端使用什麼技術、後端使用什麼技術、部署在哪裡

3 本地啟動方式

詳細的步驟說明，從 clone 專案開始，一步一步到能在瀏覽器看到畫面

4 環境需求

明確列出需要的 Node.js 版本、必要的工具（如 npm、git）

5 環境變數設定

說明需要設定哪些環境變數、API Base URL 如何設定

6 常見錯誤排查

列出你在開發過程中遇到的常見問題與解決方法

7 目前的限制

誠實說明哪些功能還沒完成、哪些是刻意不做的

三種環境的不同需求



本地開發環境

使用 `.env` 檔案儲存環境變數，這個檔案應該被 `.gitignore` 忽略，每個開發者在自己的電腦上維護自己的版本



Git 儲存庫

絕對不要提交包含真實金鑰的檔案，可以提交 `.env.example` 作為範本，讓其他開發者知道需要設定哪些變數



雲端部署環境

使用雲端平台提供的環境變數設定功能（如 Cloud Run 的環境變數、Firebase 的配置），不要把敏感資訊寫在程式碼裡

哪些設定該放在環境變數？

一定要用環境變數

- API 金鑰與密鑰
- 資料庫連線字串
- 第三方服務的認證資訊
- 加密用的 secret key
- 雲端服務的存取權杖

可以寫在程式碼中

- API 端點的網域名稱（如果是公開的）
- 應用程式的名稱與版本
- UI 相關的設定值
- 公開的功能開關

使用 Q 設計環境變數策略

請協助我設計一個「安全的环境變數使用策略」，適用於：

- 本地開發
- Git 同步
- 雲端部署 (Cloud Run / Firebase)

請說明：

1. 哪些設定該放在環境變數
2. 哪些可以寫在前端 config
3. 本地與雲端如何區分
4. 常見錯誤（例如把 key 寫死）

請用概念 + 步驟說明，不要直接貼實作碼。

這個 prompt 會讓 AI 給你一個完整的策略規劃，而不只是程式碼片段。理解概念比記住語法更重要。

建立 .env.example 檔案

在專案根目錄建立一個 .env.example 檔案，用來示範需要設定哪些環境變數，但不包含真實的值：

```
API_KEY=your_api_key_here  
DATABASE_URL=your_database_url_here  
JWT_SECRET=your_secret_here  
PORT=3000
```

這個檔案可以安全地提交到 Git，讓其他開發者知道需要設定哪些變數。他們可以複製這個檔案為 .env，然後填入自己的真實值。

什麼時候該 commit ?

→ 完成一個獨立的功能或修正

例如：新增登入表單、修正某個 bug、調整某個元件的樣式

→ 在轉換工作重點之前

當你要從開發前端切換到後端，或是從功能開發切換到 bug 修正時，先 commit 當前的進度

→ 程式碼能夠正常運作

不要 commit 會導致程式無法執行的中間狀態，確保每個 commit 都是可運行的版本

→ 每天結束工作前

即使功能還沒完成，也可以 commit 當前的進度，但要在 commit message 中說明這是進行中的工作

一次 commit 應該包含什麼？

好的 commit

- 只包含一個邏輯上的變更
- 相關的檔案一起提交
- 可以用一句話清楚描述
- 程式碼能夠正常運作

❏ 範例：新增使用者註冊功能（包含前端表單、後端 API、資料庫欄位）

不好的 commit

- 混合多個不相關的變更
- 包含大量的格式調整與實際變更
- 會導致程式無法執行
- 沒有清楚的主題

❏ 反例：更新了登入功能、修正了兩個 bug、調整了整個專案的縮排格式

常見的壞 commit 習慣

只寫「更新」或「修改」

這種 commit message 完全沒有提供任何有用的資訊，幾週後連你自己都不知道這次 commit 做了什麼

一次 commit 包含過多變更

一次改了十幾個檔案、數百行程式碼，當出問題時很難找出是哪個變更造成的

commit 會導致程式無法執行

每個 commit 都應該是可運行的狀態，這樣當需要回溯時才能找到穩定的版本

忘記處理衝突就 commit

在多人協作時，merge 後可能產生衝突，一定要先解決衝突並測試過才能 commit



推薦的 Commit Message 寫法

一個好的 commit message 應該讓人一眼就看出這次變更的目的。以下是一些實用的範例：

使用 Q 學習 Commit 最佳實踐

你是版本控制顧問。請幫我規劃一個適合新手的 Git commit 節奏與原則。

請說明：

1. 什麼時候該 commit
2. 一次 commit 裡面應該包含什麼
3. 常見的壞 commit 習慣
4. 推薦的 commit message 寫法（範例）

限制：

- 以「可回溯、可教學」為目標

讓 AI 幫你整理這些原則，並針對你的專案類型給予建議。

使用 Q 扮演「新同事」角色

請你扮演「接手這個專案的同事」。

根據我的專案結構與 README，請幫我檢查：

1. 有沒有看不懂的地方
2. 哪些步驟可能會卡關
3. 哪些資訊不夠清楚
4. 如果我要讓新手接手，最該補哪一段

請用審查清單方式回饋。

這個 prompt 會讓 AI 從一個完全陌生的角度來檢視你的專案文件，指出那些你可能忽略的問題。

交付檢查清單

1 專案結構清晰

資料夾組織方式一目了然，前端與後端明確分離，沒有混亂的檔案散落在奇怪的位置

3 README 完整且可執行

任何人照著 README 都能成功啟動專案，不需要額外的口頭說明

5 Commit 歷史有意義

每個 commit 都有清楚的 message，能夠追蹤程式碼的演進過程

2 .gitignore 正確設定

敏感資訊不會被提交，不必要的檔案（如 node_modules）被正確忽略

4 環境變數策略清楚

有 .env.example 範本，文件中說明了如何設定環境變數

6 沒有硬編碼的敏感資訊

程式碼中不包含 API 金鑰、密碼等敏感資訊

為什麼這一章比寫程式還重要？

許多初學者會覺得 Git、README、環境變數這些東西「不如寫程式有成就感」。但事實上，這些能力才是區分「會寫程式的人」與「能交付產品的開發者」的關鍵。

如果你的專案有以下任何一個問題，那麼無論程式碼寫得多好，它都還不是一個完整的產品：

- 寫完不敢給人看 — 因為擔心敏感資訊外洩，或是專案結構太亂
- 壞了不知道怎麼救 — 沒有清楚的版本記錄，無法回到穩定的狀態
- 做了卻不能交付 — 別人拿到專案後完全不知道怎麼啟動或使用

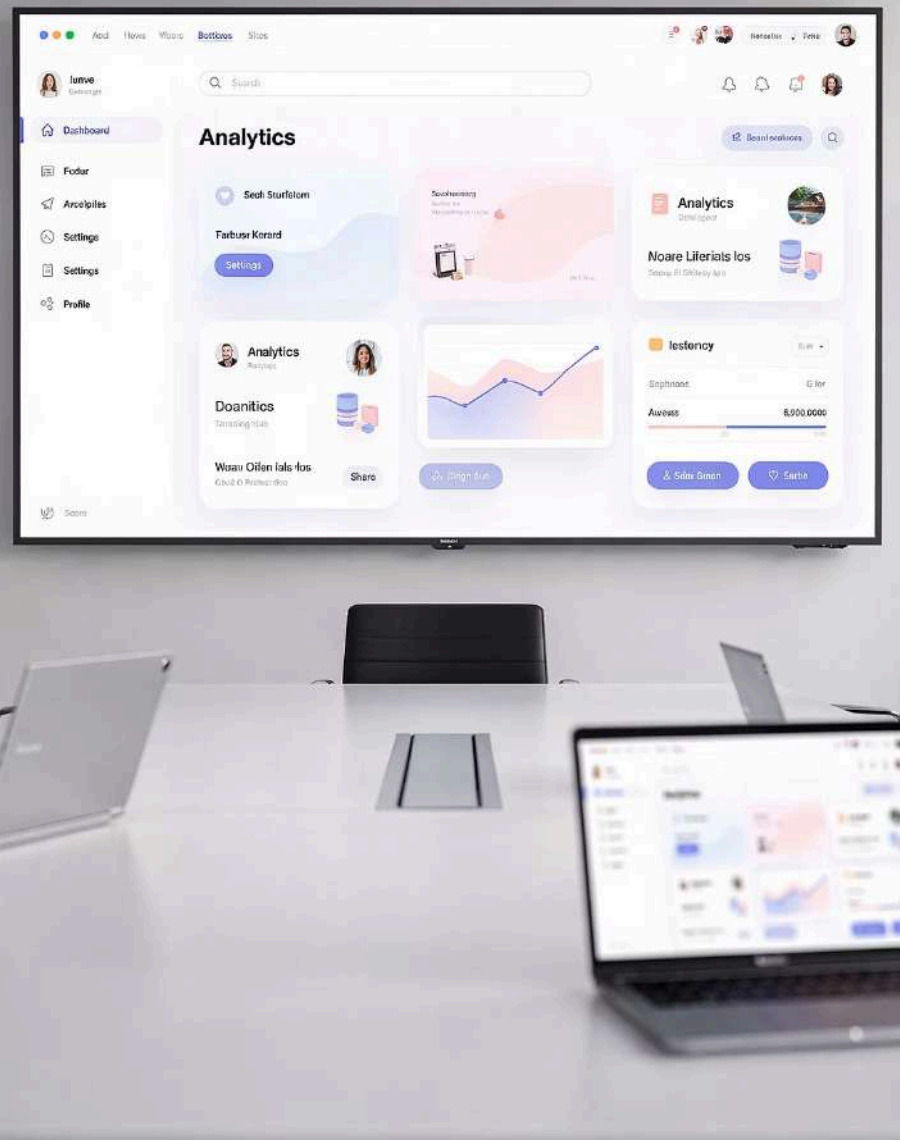
① 這一章教你的，是讓一個東西「離開你之後還活得下去」的能力。這才是真正的產品開發能力。

Chapter 7

部署與展示

把你的 Web App 變成 「能給人看的東西」

從技術實現到產品展示，讓你的作品能夠被理解與記住



常見的展示問題

「這個網站是做什麼的？我該從哪裡開始？」

「我按了這個按鈕，但不知道有沒有成功，也不知道接下來該做什麼。」

「看起來很複雜，我不太敢亂點，怕弄壞什麼東西。」

「用完了，然後呢？我應該看到什麼重點？」

這些反應都指向同一個問題：展示（Demo）不是部署的副產品，而是一個需要刻意設計的過程。

展示前必須做的「收斂」

許多初學者在展示時會犯一個常見的錯誤：想要展示所有功能。結果反而讓觀眾更加困惑，因為資訊太多、重點不清楚。

在準備展示之前，你必須做一個重要的決定：只展示一條主線流程。

這不是因為其他功能不重要，而是因為：

- 人的注意力有限，一次只能記住幾個重點
- 完整的流程比零散的功能更容易理解
- 清楚的主線能突顯產品的核心價值



展示的黃金法則

與其展示十個半完成的功能，不如完美展示一個完整的使用流程。這才能讓觀眾記住你的產品。

展示前的三個核心問題

主線流程是什麼？

從使用者打開網頁到完成一個有意義的任務，這中間的每一個步驟是什麼？最多不要超過 5 個步驟。

暫時不展示什麼？

哪些功能雖然已經實作，但在這次展示中可以先不提？把這些列出來，避免展示時分心。

觀眾應該記住什麼？

如果觀眾在看完展示後只能記住一句話，那應該是什麼？這就是你的核心訊息。

如果你無法清楚回答這三個問題，那麼展示很可能會變成一場災難。

使用 Q 定義展示目標

你是產品展示顧問。我有一個已部署的 Web App，請協助我定義「展示版 Demo 的目標」。

請幫我輸出：

1. 這個展示版要讓觀眾理解的一句話
2. 展示時唯一的一條操作流程（step by step）
3. 這次展示刻意不展示的功能清單
4. 觀眾看完後，應該留下的三個印象

限制：

- 假設觀眾不懂技術
- 假設你只有 2-3 分鐘

讓 AI 幫你整理思路，釐清展示的核心目標。記住：展示不是為了證明你做了很多功能，而是為了讓觀眾理解產品的價值。

展示流程設計範例

主線 MVP（節奏追蹤 App）

1. 開啟網頁，看到今天的任務清單
2. 選擇一個任務，開始計時
3. 完成任務，標記為完成
4. 查看今天的進度摘要

核心訊息：「一個幫助你追蹤每日任務進度的簡單工具」

匿名回饋牆

1. 開啟網頁，看到已有的回饋訊息
2. 點擊「新增回饋」
3. 輸入匿名回饋並送出
4. 立即看到自己的回饋出現在牆上

核心訊息：「一個讓團隊能夠安全分享真實想法的平台」

使用 Q 檢視展示 UI

你是 UI/UX 檢視顧問。這是一個要展示給外人看的 Web App。

請幫我檢視目前的畫面,並提出「最小修改建議」,讓它更適合 Demo。

請聚焦在：

- 初次打開是否看得懂？
- 按鈕與文字是否有引導性？
- 是否有任何會讓人困惑的元素？

限制：

- 不重寫整個 UI
- 只給「必要調整建議」

展示 UI 的關鍵調整點



清楚的標題與說明

在頁面頂端加上一句話說明這個 App 是做什麼的，不要假設使用者會自己猜測



示範資料的存在

不要讓使用者面對空白頁面，預先放入一些示範資料，讓他們看到「成功的樣子」



引導性的按鈕文字

把「送出」改成「新增我的回饋」，把「確定」改成「開始計時」，讓按鈕本身就能說明動作



明確的操作回饋

當使用者完成一個動作後，要有清楚的視覺回饋（如成功訊息、狀態變化）讓他們知道操作成功了

最小修改原則

記住一個重要原則：不要為了展示而完全重做 UI。你的目標是做最小但最有效的調整，讓展示更順暢。

以下是一些不需要做的事：

- 完全重新設計視覺風格
- 加入複雜的動畫效果
- 實作完整的響應式設計
- 追求像素級的完美對齊

你應該聚焦在：

- 關鍵的文字說明與引導
- 按鈕文字的清晰度
- 操作回饋的明確性
- 減少困惑的元素

❗ 目標是「可用」而非「完美」。展示的成功與否取決於觀眾能否理解與操作，而非視覺設計是否完美。

90 秒展示說詞結構

1 開場（10 秒）

一句話說明這是什麼產品

3 操作（40 秒）

現場示範主線流程

2 情境（20 秒）

描述一個真實的使用情境

4 結尾（20 秒）

強調產品的核心價值

使用 Q 準備展示說詞

你是 Demo 簡報教練。請幫我為這個 Web App 準備一段「90 秒展示說詞」。

請包含：

1. 開場一句話（在幹嘛）
2. 一個真實使用情境
3. 現場操作的三個重點步驟
4. 結尾一句「這個東西的價值」

限制：

- 對象是非工程背景
- 用白話，不用技術名詞

展示說詞範例

開場：「這是一個幫助團隊收集匿名回饋的工具。」

情境：「想像你們團隊剛完成一個專案，想要了解大家真實的想法，但又擔心在會議上直說會有壓力。這時候就可以用這個回饋牆。」

操作：「首先，每個人打開這個網址，就能看到一面回饋牆。接著，點擊這裡新增你的匿名回饋。寫完送出後，所有人都能立即看到，但沒人知道是誰寫的。」

結尾：「這樣一來，團隊就能收集到更真實、更有價值的回饋，而不是只聽到那些『安全的意見』。」

避免技術術語

✗ 不好的說法

「這個 App 使用 React 前端框架和 Node.js 後端，透過 RESTful API 進行資料交換，並部署在 Google Cloud Run 上。」

問題：太多技術細節，觀眾會失焦，無法理解產品的實際價值。

✓ 好的說法

「這個 App 讓你能在網頁上快速新增與查看團隊回饋，而且完全匿名，不需要登入或下載任何東西。」

優點：聚焦在使用者體驗與價值，用日常語言描述。

使用 Q 設計測試資料

你是 Demo 測試資料設計師。請幫我設計一組「展示專用測試資料」。

需求：

- 可以快速帶出產品重點
- 不需要太多資料
- 每一步操作都有明確效果

請輸出：

- 測試資料內容
- 建議操作順序
- 每一步應該讓觀眾看到什麼

好的測試資料特徵

- **真實但簡化**

使用真實的名稱與情境，但內容要簡短清楚。例如：
「完成產品設計文件」而非「test task 1」

- **有對比性**

如果要展示狀態變化，測試資料應該包含不同狀態的範例（如：已完成、進行中、未開始）

- **數量適中**

不要放太多資料讓畫面雜亂，也不要太少讓功能無法展現。通常 3-5 筆是最適合的

- **支持故事線**

測試資料應該能串起你的展示流程，而不是一堆不相關的隨機內容

展示前檢查清單

1 網址可正常存取

在不同裝置與網路環境測試，確保網址能夠正常開啟，載入速度合理

3 測試資料已準備

展示用的測試資料已經建立，不會在操作時才發現沒有資料可用

5 主要流程順暢

從頭到尾走一次展示流程，確保每個步驟都能正常執行，沒有卡住的地方

2 API 正常運作

測試所有展示流程會用到的 API 端點，確保回應時間可接受，沒有錯誤

4 沒有開發用錯誤訊息

檢查 console 不會顯示一堆紅色錯誤，使用者不會看到技術錯誤訊息

6 備援計畫準備好

準備好螢幕截圖或錄影，萬一現場出問題時能夠應急

不同類型 Web App 的展示重點

1 主線 MVP（節奏追蹤）

重點：狀態變化，而非資料量

展示如何從「未開始」到「進行中」再到「已完成」，讓觀眾看到進度的變化。不需要展示大量任務，3-4 個就足夠。

避免：花時間解釋背後的演算法或計算邏輯

2 匿名回饋牆

重點：多人看到同一份資料

重點是展示「即時同步」的概念 — 一個人新增回饋，其他人立刻看到。可以用兩個瀏覽器視窗來展示這個效果。

避免：深入解釋即時同步的技術細節

3 決策型 Web App

重點：前端不知道規則

強調「前端只負責收集輸入與顯示結果，判斷邏輯完全在後端」這個架構特點。可以展示輸入不同資料，系統做出不同判斷。

避免：公開解釋判斷規則的細節

本章完成標準與展示的真正價值

完成標準

你已經完成 Chapter 7，如果你能做到：

- 把網址傳給朋友，他不用問你就能操作
- 你能在 90 秒內完整展示一條流程
- 對方看完能說出「這個產品在幹嘛」

如果這三件事成立，代表你的 Web App 已經「可展示」了。

展示的真正價值

展示能力是產品能力的一部分。因為產品不是寫給自己用的。如果你無法清楚展示，代表：

- 產品定位不清楚
- 使用流程不清楚
- 價值主張不清楚

展示，是逼你把這三件事講清楚。當你能用 90 秒讓一個陌生人理解你的產品，那才是真正的完成。



Chapter 8

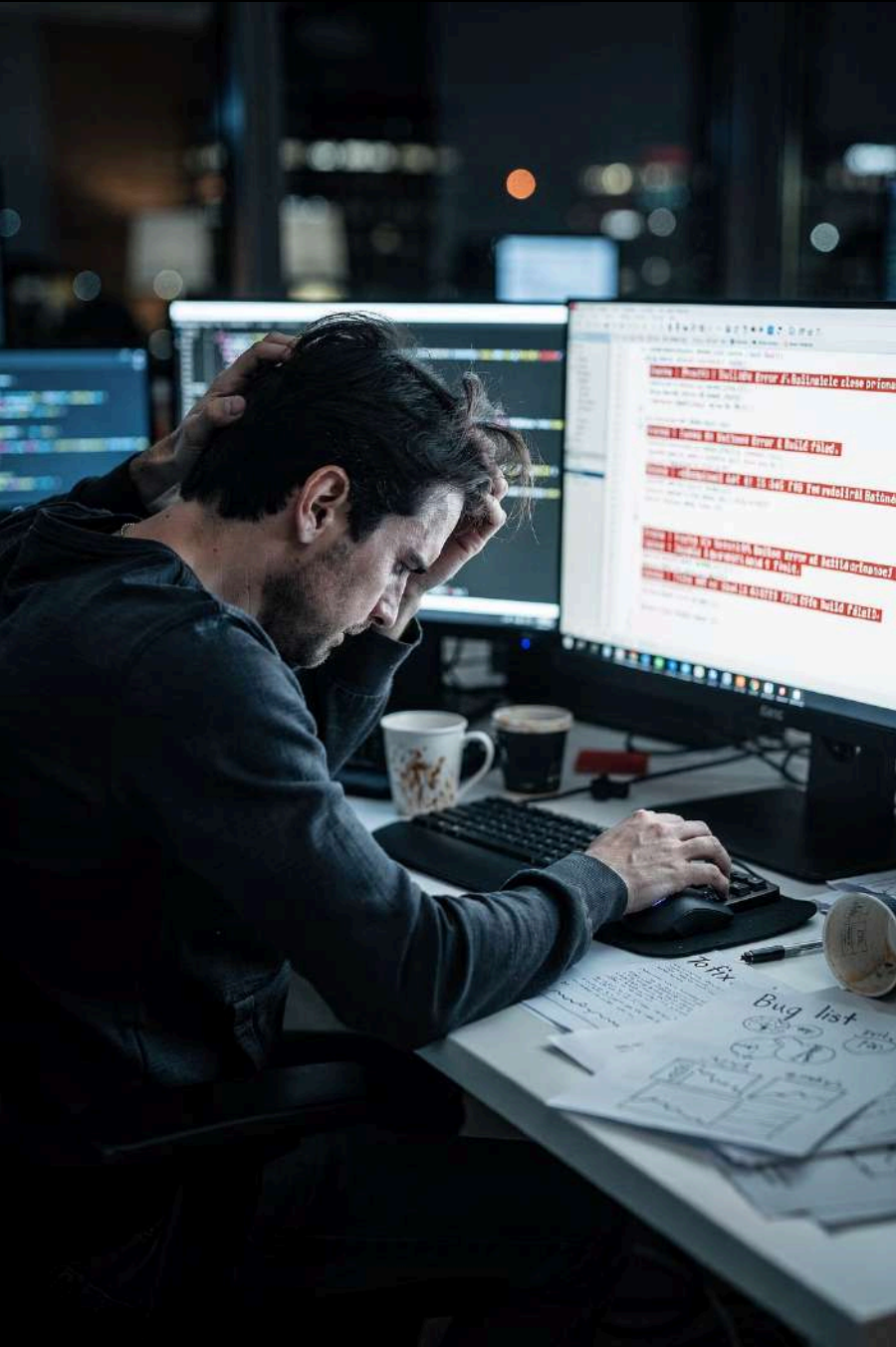
Vibe Coding

常見翻車點與修正方式

當你開始大量使用 AI 輔助開發工具後，很容易陷入一個危險的錯覺：「反正 AI 會幫我補齊」。

這種過度依賴會讓專案逐漸失控：程式碼越來越多，但你卻越來越說不清楚系統在做什麼。

本章將深入探討 Vibe Coding 最容易翻車的場景，以及如何精準修正。



Vibe Coding 真正的風險所在

1 程式碼膨脹

專案檔案快速增加，但缺乏清晰的組織結構

2 失去掌控感

無法清楚說明每個模組的職責與互動關係

3 除錯困難

出問題時不知道該從哪一層開始檢查

Vibe Coding 的風險不在於「寫錯」，而在於「失去結構感」。當你無法清楚描述系統架構時，即使程式能跑，專案也已經處於危險邊緣。

翻車類型一：指令太大，一次要太多

常見症狀

- 一次丟給 Q 一整段複雜需求
- Q 回傳一大包程式碼
- 不知道該貼到哪個檔案
- 不確定該修改哪些部分
- 不清楚該刪除什麼內容

為什麼會翻車

因為你跳過了「拆解任務」這個關鍵步驟，直接要求 AI 給你完整解決方案。這就像是要求建築師在沒有藍圖的情況下直接蓋房子。

AI 會盡力滿足你的要求，但當需求範圍不明確時，它產出的程式碼往往會包含許多你不需要的延伸功能，或是跨越多個職責邊界。

修正方式：把指令拆成「只做一件事」

📄 精準指令範本

請你只做一件事：_____

限制條件：

- 不要延伸到其他功能
- 不要幫我優化或加入新需求
- 只處理我明確指定的範圍

判斷標準非常簡單：如果 Q 回傳的程式碼讓你不知道該放在哪個檔案，那這個指令就太大了。

良好的指令應該能明確對應到一個檔案、一個函式、或一個元件。當你能清楚預期程式碼的歸屬位置時，就代表任務拆解得當。

實例對比：好指令 vs 壞指令

✗ 壞指令範例

「請幫我實作一個完整的使用者系統，包含註冊、登入、權限管理和個人資料頁面」

問題：範圍太大，跨越多個模組，無法預期輸出結構

✓ 好指令範例

「請在 UserService.ts 中新增一個 validateEmail 函式，只負責驗證 email 格式是否正確」

優點：範圍明確，職責單一，容易整合

翻車類型二：沒有 API 契約就開始寫

1 前端先寫

假設 API 格式開始開發 UI

2 後端補不回來

實際 API 結構與前端預期不符

3 兩邊都在猜

最終雙方都不確定對方的實作細節

這是最常見也最致命的協作問題。當前後端沒有明確的介面契約時，你實際上是讓 AI 同時扮演「設計師」與「工程師」兩個角色，但你自己卻沒有站在任何一方進行把關。

修正方式：先叫 Q 只寫「契約文件」

❏ 契約優先指令範本

請你現在不要寫任何程式碼，只幫我產出 API 契約文件。

內容必須包含：

- 所有 endpoints 的完整路徑
- 每個 endpoint 的 request / response JSON 格式
- 每個欄位的型別與意義說明
- 可能的錯誤回應格式

記住這個原則：只要契約沒定完，任何程式碼都只是暫時的。

先確立介面契約能讓前後端基於相同的認知進行開發，避免後期大規模重構。這個步驟看似增加時間，實際上會大幅降低整體開發成本。

API 契約的關鍵要素

1 Endpoint 定義

明確的 HTTP 方法與路徑，例如 `POST /api/users/login`

2 Request 格式

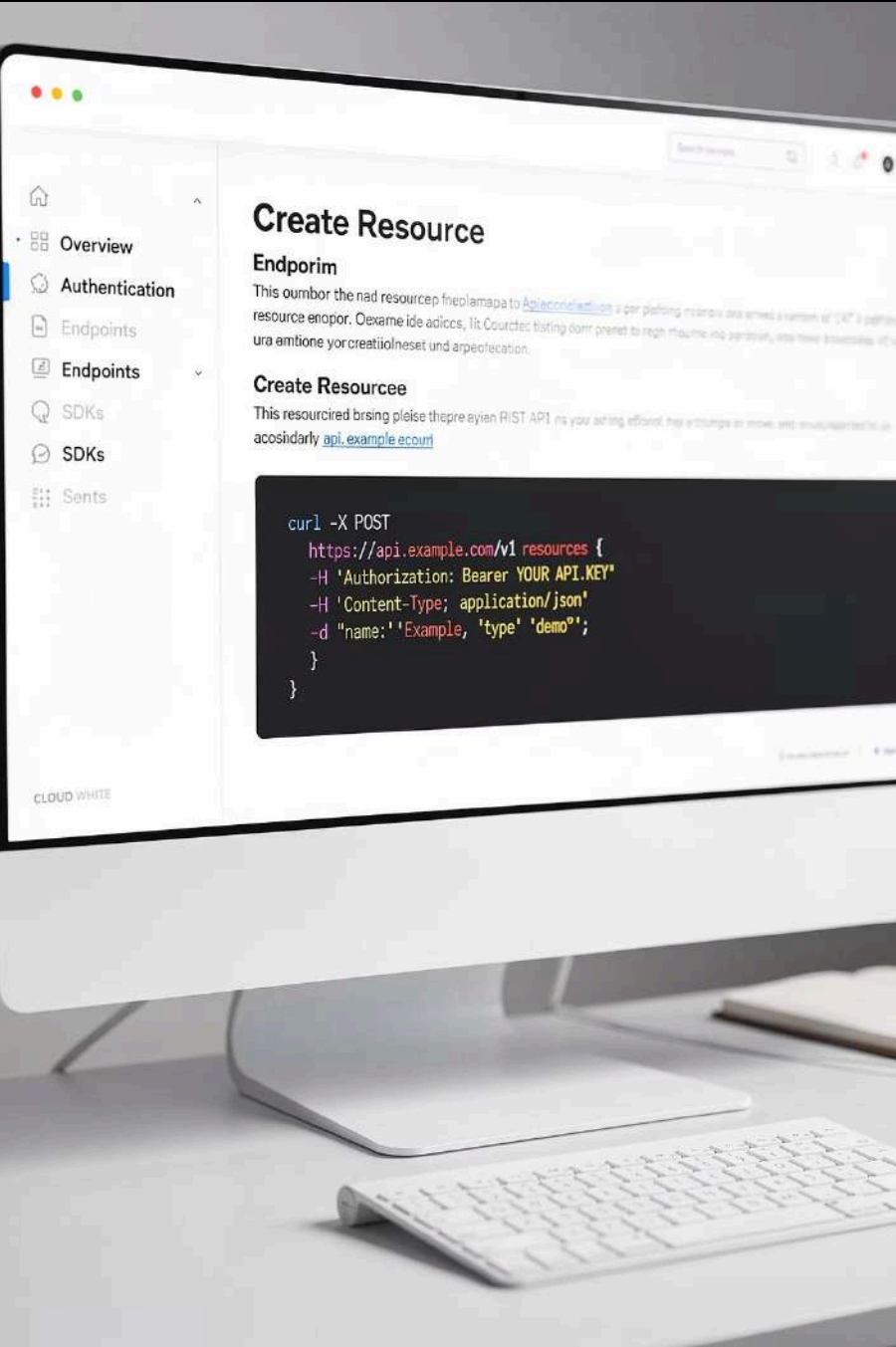
清楚的輸入參數結構、型別與驗證規則

3 Response 格式

成功與失敗情境的回應結構，包含狀態碼

4 欄位說明

每個欄位的業務意義、可選性與預設值



翻車類型三：前端開始偷做後端的事

常見症狀清單

- status 狀態在前端計算
- decision 決策邏輯在前端判斷
- 商業規則散落在 UI 元件裡
- 資料處理邏輯寫在畫面層
- 權限檢查在前端進行

為什麼會這樣翻車

因為「這樣比較快看到結果」——在前端直接處理邏輯能讓畫面立刻動起來，不需要等待後端 API。

但你實際上是在犧牲產品結構，將業務邏輯與展示邏輯混在一起，讓系統變得脆弱且難以維護。

修正方式：強制 Q 檢查責任歸屬

❏ 責任檢查指令範本

請你檢查目前這段功能設計，逐條標示：

- 哪些是前端責任（UI 呈現、使用者互動、畫面狀態）
- 哪些是後端責任（資料驗證、商業邏輯、狀態管理）

如果有責任放錯地方，請直接指出並說明原因與修正建議。

關鍵原則：如果 Q 指出「這段不該在前端」，你要相信它的判斷，不要為了暫時的便利而硬拗。

長期來看，正確的職責劃分會讓系統更容易擴展、測試和除錯。

前後端職責劃分原則



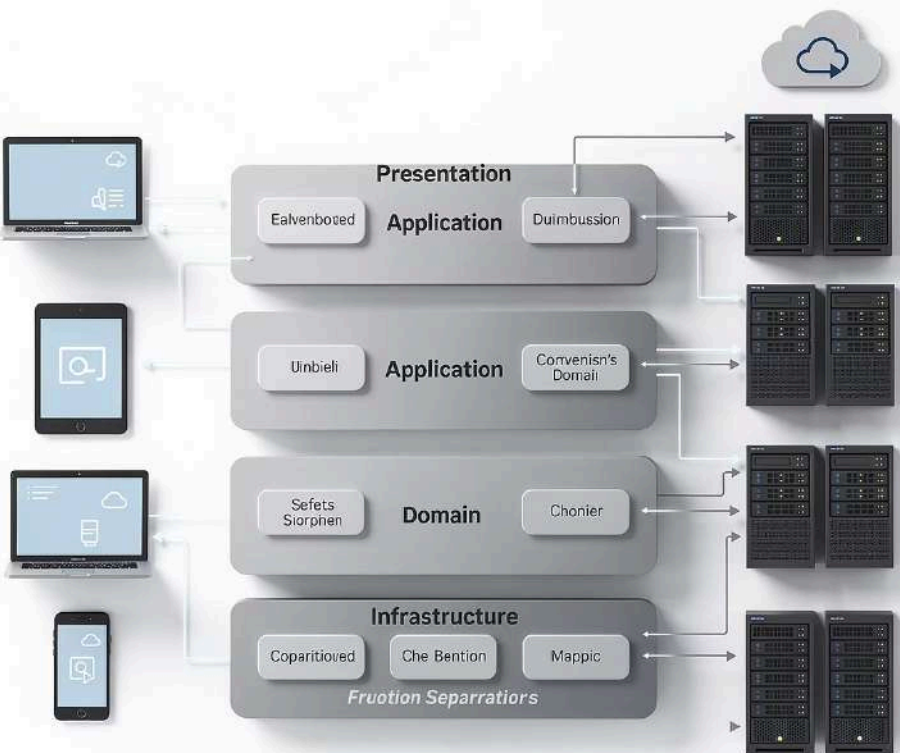
前端責任

- UI 呈現與互動回饋
- 表單輸入初步驗證
- 畫面狀態管理
- 路由與導航邏輯



後端責任

- 商業邏輯與決策
- 資料驗證與處理
- 權限與安全控制
- 狀態計算與持久化



翻車類型四：改一點東西，整個系統崩

這是最讓人挫折的場景：你只是想修改一個小細節，結果觸發連鎖反應。

1 修改一個欄位

改變資料結構中的一個屬性名稱

2 API 全部掛掉

後端回傳格式與前端預期不符

3 前端報錯不明

錯誤訊息模糊，難以定位問題

為什麼會翻車？因為你沒有設定「修改邊界」。AI 在修改程式碼時，可能會「順便」調整其他相關部分，導致影響範圍遠超預期。

修正方式：用 Q 做 Atomic Edit

❏ 原子化修改指令範本

我要修改一個非常小的地方：_____

嚴格限制條件：

- 只允許修改指定的檔案
- 只修改與此需求直接相關的程式碼
- 不要順手重構其他地方
- 不要「改善」周邊程式碼

修改完成後，請列出：

1. 你改了哪幾行（具體行號）
2. 為什麼要改這些行
3. 有哪些地方可能需要對應調整

這個指令是避免 AI「順便幫你搞更大包」的關鍵防護措施。

Atomic Edit 的核心思維

原子化修改的好處

- 影響範圍可控
- 容易進行版本比對
- 出問題時容易回溯
- 降低意外副作用
- 保持程式碼穩定性

每次修改都應該像外科手術一樣精準：知道切入點在哪裡，知道影響範圍有多大，知道如何驗證結果。

當你養成這個習慣後，即使專案規模擴大，你依然能維持對系統的掌控感。這就是專業開發者與業餘愛好者的關鍵差異。

翻車類型五：錯了，但不知道錯在哪一層

系統出現問題時，最困難的往往不是解決問題本身，而是找出問題發生在哪一層。

常見錯誤症狀

- 404 錯誤 — 找不到資源
- CORS 問題 — 跨域請求被阻擋
- 資料是空的 — API 有回應但內容異常
- 無法判斷來源 — 不確定是前端、後端還是部署環境的問題

這種狀況特別容易讓人陷入「猜測 + 嘗試」的低效循環，浪費大量時間在錯誤的方向上。

修正方式：強制分層 Debug

❏ 系統化除錯指令範本

我現在有一個錯誤現象：_____

請你不要直接給我解法，而是依序幫我檢查：

1. 前端層可能的問題點
2. API / 後端層可能的問題點
3. 部署 / 環境層可能的問題點

針對每一層，請提供：

- 具體的檢查方式（如何驗證）
- 預期看到的正確結果
- 如何排除該層的問題

這個方法會讓你重新取得「系統視角」，從整體架構角度理解問題。

分層除錯的黃金流程

1

前端檢查

確認請求是否正確發出、參數格式是否符合預期

2

API 檢查

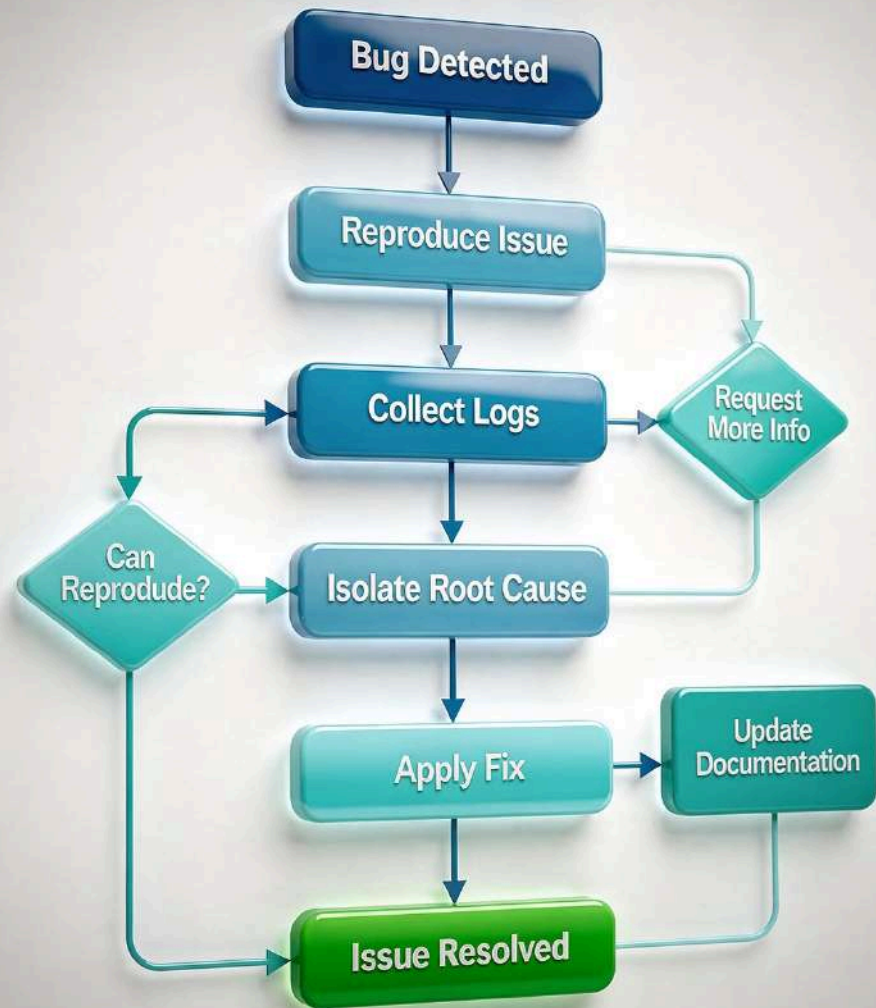
驗證後端是否收到請求、回應格式是否正確

3

環境檢查

確認部署設定、環境變數、網路連線狀態

按照這個順序逐層排除，能大幅提升除錯效率。每一層確認無誤後才往下一層移動，避免在錯誤的層級上浪費時間。



翻車類型六：Q 寫得越來越不像你

混亂症狀觀察

- 檔案風格不一致
- 命名規則混亂
- 註解語氣差異大
- 每次生成像不同人寫的
- 程式碼缺乏統一感

根本原因分析

因為你沒有給 AI 明確的「風格約束」。每次對話都是獨立的上下文，AI 會根據當下的提示詞調整輸出風格。

沒有一致的風格指引，專案程式碼會逐漸變成大雜燴，後期維護成本會急遽上升。

修正方式：把風格寫進指令

📄 風格約束指令範本

在接下來所有產出中，請嚴格遵守以下風格規範：

命名規則：

- 變數使用 camelCase
- 元件使用 PascalCase
- 常數使用 UPPER_SNAKE_CASE

檔案結構：

- 相同功能的檔案放在同一資料夾
- 測試檔案與原始檔案並列

註解語氣：

- 使用繁體中文
- 簡潔明確，避免冗長解釋

重要：如果你覺得某個規範不適合當前情境，請先詢問我，不要自行更改風格。

建立專案風格指南的實務建議



建立 Style Guide 文件

在專案根目錄建立一份 `STYLE_GUIDE.md`，記錄所有風格規範。每次請 AI 生成程式碼時，可以直接引用這份文件。



使用程式碼範本

準備好標準的檔案範本，包含註解格式、`import` 順序、基本結構等。讓 AI 參考範本產生一致的輸出。



Code Review 檢查點

每次接收 AI 生成的程式碼後，用固定的檢查清單確認風格一致性，逐漸培養辨識能力。

AI 很會配合，但前提是你有說清楚

Developer Q 的強大之處在於它的適應性，但這也意味著你必須明確告訴它你要什麼。

AI 不會主動統一風格

除非你明確要求，否則它每次都會根據當下語境產生不同風格的程式碼

風格規範需要持續強調

在每次對話開始時重申風格要求，或將規範放入系統提示中

一致性是你的責任

AI 提供材料，但維持專案一致性的責任仍在於開發者本身

本章的核心不是避免錯誤 而是避免失控

錯誤不可怕，失控才可怕。

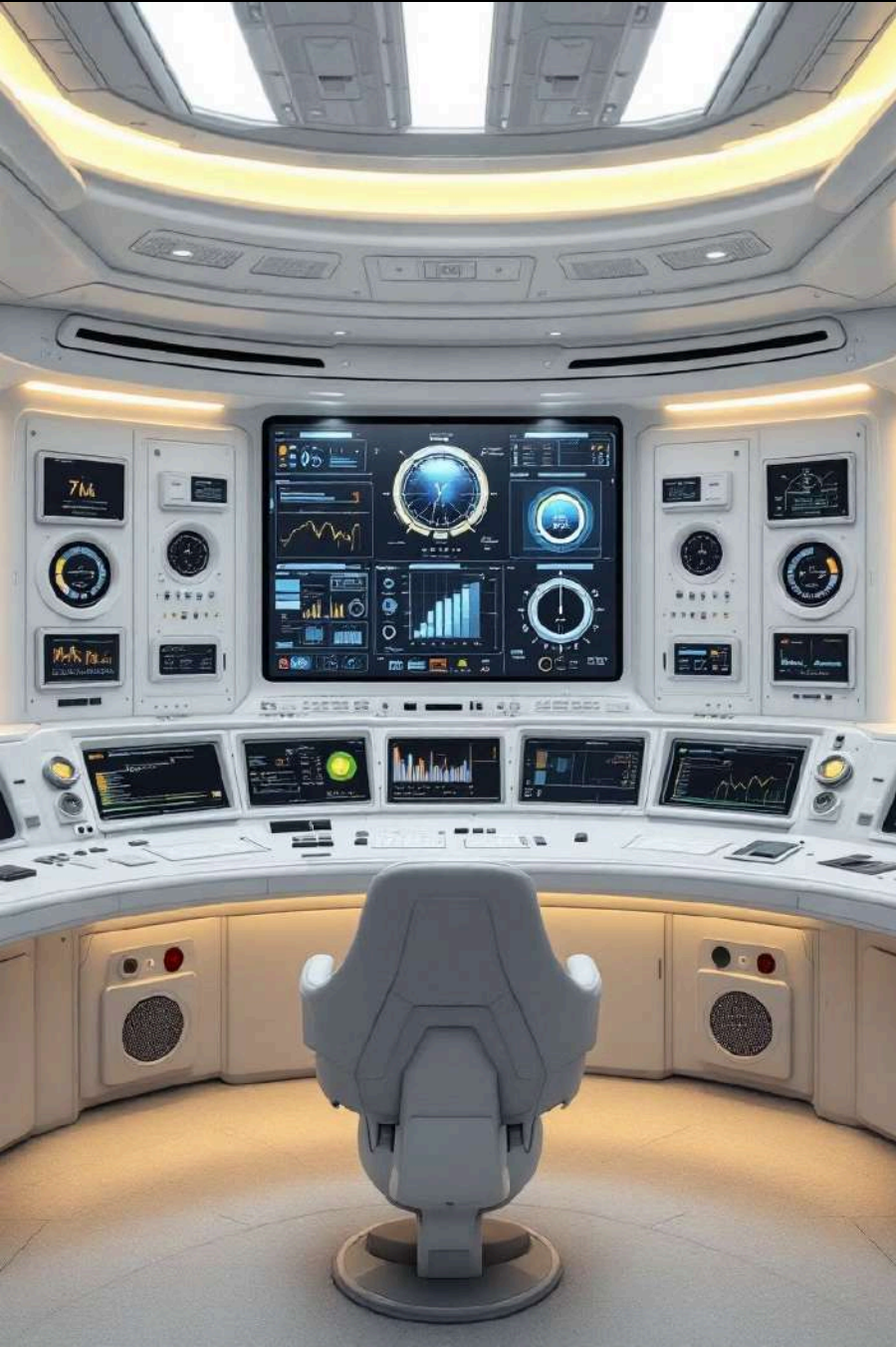
寫程式一定會出錯，這是開發過程的正常現象。重要的是當錯誤發生時，你是否能：

- 快速定位問題來源
- 理解錯誤的影響範圍
- 知道如何修正而不擴大災情
- 從錯誤中學習並改進流程

Vibe Coding 真正要學的不是
「讓 AI 幫你寫程式」，而是：

你還能不能清楚知道現在在做什麼。

當你失去對系統的理解時，即使程式碼都能跑，你也已經失去控制權。這時候的專案就像一顆不定時炸彈，隨時可能因為小修改而全面崩潰。



Chapter 9

商業評估

產品要怎麼活下來

做到這一章，你已經證明了三件重要的事：你能用 AI 把產品做出來、能完成前後端整合與部署、不只是會寫 Demo 而已。

但接下來真正的問題會變成：這個東西，怎麼在現實世界中存在？



技術不是問題，商業結構才是

AI Coding 產品最大的誤判，往往不在技術實力，而是商業結構想錯層級。

技術可行性 ✓

你已經證明產品能做出來，功能可以運作，系統架構合理

商業可行性 ？

但這個產品能否找到付費使用者？成本結構是否健康？能否持續營運？

很多優秀的技術專案死在商業模式不明確、成本控制失敗、或是找不到真正願意付費的使用者。接下來我們要面對的，就是這些現實問題。

AI Coding 產品最常見的三個錯誤期待

錯誤一

「功能做出來，自然會有人付費」

事實是：沒有人會為「功能」付費，只會為「減少風險或時間」付費。

錯誤二

「我可以先免費，之後再想辦法」

AI 產品一旦牽涉到 API 成本，「免費」其實等於「你在補貼陌生人」。

錯誤三

「這個很厲害，一定有人買」

AI Coding 很容易「對工程師很厲害」，但對使用者只是「看不懂」。

AI Coding 產品的五種常見商業模式

以下不是理論模型，而是你現在這種產品最可能走的實際路徑。了解這些模式能幫助你更早做出正確的策略選擇。

1 工具型 SaaS

單一功能、解決特定問題、月費或使用量計費

2 BYOK 平台型

使用者自備 API Key，你收取平台使用費

3 專案顧問型

客製化解決方案、一次性專案費用

4 內容工具混合

工具搭配教學內容、課程或會員制

5 展示作品集

不直接變現，用於展示能力與爭取機會

模式一：工具型 SaaS

產品特徵

- 一個明確的核心功能
- 解決一個小但重複的問題
- 使用者能快速理解價值

適合的產品類型

- 決策型 Web App
- 自動判斷、篩選工具
- 資料整理與分析工具

付費邏輯

月費或年費訂閱制，依使用量或功能層級限制

風險點

- 使用頻率可能很低
- 用完就走，留存率差
- 競爭者容易複製

判斷關鍵：這個工具，使用者一個月會開幾次？

模式二：BYOK 平台型（AI 功能層）



核心概念

核心價值不是 AI 模型本身，而是「流程設計、介面體驗、判斷邏輯」



適合產品

AI 輔助決策工具、AI 分析平台、AI 教練型產品



付費模式

使用者自己提供 API Key，你收取平台功能使用費

最大優勢：成本風險極低，可以快速驗證產品價值。這是課堂 MVP 最推薦的商業路線。

模式三：專案型／顧問型

商業模式說明

不是賣標準化產品，而是賣「這套系統幫你解決什麼問題」的完整解決方案。

適合的情境

- 企業內部系統（節奏追蹤、決策工具）
- 特定產業的客製化需求
- 部門專用系統

付費結構

一次性專案費用，可能包含後續維護合約

現實優點

- 最快變現 — 不需要等規模化
- 客戶清楚願意付多少錢
- 不需要承擔營運成本

很多 AI Coding MVP 最適合先走這條路，累積資金與經驗後再考慮產品化。

模式四：內容 × 工具混合型

產品定位

工具本身不是主要賣點，「怎麼用、怎麼想、如何應用」的知識內容才是核心價值

適合對象

教學型 AI 工具、創作者輔助系統、需要學習曲線的專業工具

付費方式

線上課程、會員訂閱制、工具模板包、進階教學內容

如果你本身是創作者、講師或意見領袖，這條路會比純粹賣工具更合理。

模式五：展示型（不賣產品）

這不是失敗，而是一個清楚且務實的策略選擇。

1 展示能力

證明你能用 AI 快速打造完整產品

2 爭取機會

透過作品獲得合作、工作或專案邀約

3 累積曝光

建立個人品牌與技術聲譽

很多 AI Coding 作品，最健康的結局就是停在這裡——不強求變現，而是讓作品為你打開更好的機會之門。

用三個問題快速篩選 商業可能性

① 關鍵自我評估問題

請對你的 Web App 誠實回答以下三個問題：

1. 如果今天有 10 個人用，你會緊張嗎？
(檢驗系統穩定性與成本控制)
2. 如果有 100 個人用，你會破產嗎？
(檢驗成本結構是否可擴展)
3. 如果有 1 個人願意付錢，你敢收嗎？
(檢驗商業化準備程度)

AI 成本結構：你一定要算的不是營收

AI 產品最常見的致命陷阱：用量一上來，成本先爆炸。

\$0.05

每次呼叫成本

看似很低，但乘以使用次數後會非常
驚人

20

每日使用次數

一個活躍使用者平均每天會呼叫 AI 幾
次？

100

使用者數量

當使用者成長到這個數字，你的月成
本是多少？

你至少要清楚知道：每一次 AI 呼叫大概多少錢、一個使用者一天會用幾次、你是不是在補貼「亂用的人」。

如果你現在不想面對這件事，那 BYOK 就不是選項，而是必要條件。

用 Q 幫你選「最適合現在的商業定位」

📄 商業模式評估指令範本

你是 AI 產品策略顧問。我有一個 AI Coding Web App，請你協助我判斷「最適合現在階段的商業模式」。

請依序分析：

1. 這個產品目前最像哪一種類型（工具 / BYOK / 專案 / 展示）
2. 為什麼不適合其他類型
3. 如果要商業化，最小可行的第一步是什麼
4. 哪些風險是我現在不該碰的

重要：請用「保守建議」而不是理想化版本回答。

這個提示的目的不是讓你興奮，而是讓你不要亂衝。

真正想讓你帶走的商業觀念 不是比誰做得多 而是比誰知道什麼時候不要做

AI Coding 產品的成功關鍵，不在於功能有多豐富、技術有多先進，而在於你能否清楚辨識：

- 哪些功能現在該做
- 哪些成本現在不該承擔
- 哪些期待現在不切實際
- 何時該停、何時該轉、何時該堅持

結課總結：你現在具備的三個稀缺能力

能把 AI 變成系統

不只是呼叫 API，而是將 AI 能力整合進完整的產品架構中，建立可維護、可擴展的系統

能判斷責任與風險

清楚知道哪些職責該放在哪一層、哪些風險現在不該承擔、如何在速度與品質間取得平衡

能選擇「停、走、轉彎」

不盲目追求完美或規模，而是根據現實條件做出務實的策略選擇，知道何時該停下來重新評估

這三件事，比任何一個爆紅 Demo 都更有價值。

從這裡開始的實踐建議



先確立定位

用本章的三個問題評估你的產品，選擇最適合當下的商業模式，不要貪心想全拿



算清楚成本

精確計算 AI API 使用成本，設定合理的使用限制，避免補貼陌生人



找到第一個使用者

與其追求很多人知道，不如讓一個人真正願意用並給你反饋



持續迭代優化

根據真實使用情況調整產品，保持系統的可控性與可維護性

你已經準備好了

完成今天的學習，你已經不只是「會用 AI 寫程式的人」，而是具備以下完整能力的開發者：

- 能有效駕馭 AI 工具，而非被工具牽著走
- 能在翻車前辨識風險，快速修正問題
- 能評估商業可行性，做出務實選擇
- 能維持系統掌控感，保持長期可維護性

接下來的路，是把這些能力應用到真實專案中，持續精進、謹慎前行。

記住：比起做出驚人的 Demo，更重要的是做出能持續運作、能真正解決問題、能在現實中存活的产品。

